

Comandos de Linux

Revisão e introdução a novos comandos

Assuntos

■ Revisão

■ Comandos novos

■ Atividade

Revisão

Comando	Função
ls	Lista todos os arquivos e pastas do diretório atual -l → mostra detalhes como permissões, dono, tamanho e data dos itens listados -a → inclui arquivos ocultos na listagem
mkdir _	Cria um novo diretório com nome _
cd	“Change Directory” é utilizado para acessar e sair de diretórios dentro do próprio terminal (“cd ..” ; “cd /usr/share/wordlists” ; “cd”)
mv	Move ou renomeia um arquivo Uso → “mv [origem] [destino]”

Revisão

Comando	Função
cat	Exibe conteúdo ou concatena arquivos “cat a.txt” → Exibe o conteúdo que a.txt possui “cat a.txt b.txt > ab.txt” → Concatena os arquivos a.txt e b.txt em ab.txt
rm	Remove arquivo ou diretório do sistema “rm a.txt” “rm -r pasta” → remove recursivamente o diretório e tudo que há dentro dele NUNCA FAÇA rm -rf / As flags recursive e force irão apagar sem perguntar tudo o que há depois de /, ou seja, o sistema todo 💀
nano	Cria ou abre um arquivo diretamente no terminal “nano a.txt” → cria o arquivo a.txt, ou, caso ele já exista no diretório, o abre

Comandos novos



grep

Ferramenta de busca de texto

Flags interessantes:

- `-i` Ignora diferenças entre maiúsculo e minúsculo
- `-r` Busca recursivamente por uma palavra
- `-c` Conta quantas vezes a palavra aparece no arquivo
- `-o` Mostra somente a palavra buscada

Exemplos:

- `grep -i "password" a.txt` → Procura pela palavra password dentro de a.txt
- `grep "password" *.txt` → procura em todos os arquivos txt do diretório pela palavra password

grep 🕒

Usos mais avançados:

```
“grep -oE '\b[a-zA-Z]{5}\b' a.txt > result.txt”
```

A flag `-E` indica que usaremos **regex** (expressão regular) para filtrar uma palavra dentro de `a.txt`

`'\b _ \b'` indica os limites da palavra (começo e fim)

`[a-zA-Z]` busca todas as palavras com qualquer letra minúscula(`a-z`) e maiúscula(`A-Z`)

`{5}` indica o tamanho da palavra que buscamos (quantidade de caracteres)

`> result.txt` cria um arquivo com o resultado da filtragem

| (Pipe)

Ferramenta para encadeamento de comandos

Basicamente manda a saída de um comando como entrada para o outro

Exemplos:

“`cat settings.txt | grep "config" | sort`” → Filtra e ordena todas as linhas que possuam a palavra config em settings.txt

“`cat *.log | grep "error"`” → procura em todos os arquivos log pelas linhas que possuam a palavra error

| (Pipe)

Usos mais avançados: □

😵 “grep "Failed password" /var/log/auth.log | grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' | sort | uniq | head” → Procura no arquivo auth.log quantas vezes cada IP tentou um login e falhou

- sort → ordena os IPs
- uniq → remove linhas duplicadas
- uniq -c → conta quantas vezes aparece
- head/tail → head exibe somente as primeiras linhas da filtragem. Já tail exibe somente as ultimas (pode ser head -n 5 para mostrar os 5 primeiros)

| (Pipe)

Extra hehe

“`history | grep -E “nmap|hydra|john|sqlmap”`” → Vê se alguém já usou algum dos comandos citados no terminal

diff

Compara o conteúdo de dois arquivos

“`diff [arquivo1] [arquivo2]`” → Compara os dois arquivos e mostra as mudanças feitas.

Exemplo:

Um usuário malicioso invade o seu sistema e modifica arquivos, mas com muita sorte você tem um backup . Com o comando diff você pode analisar as mudanças que ele fez no seu arquivo para se defender de futuros hackers



chown e chmod

Chown → define quem é o dono do arquivo/diretório

Chmod → define quais ações pode fazer no arquivo/diretório

“`sudo chown user:group a.txt`” → define user como dono do arquivo

“`sudo chmod 770 a.txt`” → define permissões no arquivo

Valores de chmod:

Leitura (r)= 4

Escrita (w)= 2

Execução (x)= 1

Exemplo:

7 (Leitura, escrita e execução) para o dono

7 (Leitura, escrita e execução) para o grupo

0 (Sem permissões) para os outros

strings

Extraí informações legíveis de arquivos

Muitas vezes um simples `cat` consegue extrair as informações de arquivos simples, porém, o `strings` consegue organizar melhor a forma como é mostrado

“`strings arquivo.bin`” → extraí informações de `arquivo.bin` de forma legível

SU

Realiza troca de usuário no próprio terminal

“**adduser user**” → Cria um novo usuario

“**su – user**” → Realiza a troca de usuário para user

Hipoteticamente, após conseguirmos credenciais, podemos invadir a máquina e acessar os diferentes usuários através desse comando 🐱💻

Atividade 🙈

1) Crie os arquivos (.log) com o conteúdo passado e:

- a) Modifique as permissões para que apenas o dono do arquivo consiga ler e escrever.
- b) Houve alguma alteração de um arquivo para o outro? Se sim, qual foi ela?
- b) Qual é o IP que mais acessa usuários? Quantos acessos bem sucedidos ele possui?

Agradeco!